

SIG WEB SITE

Audit de conformité au cahier des charges, de sécurité OWASP Top 10 et de qualité logicielle : ce qui est fait, ce qui est partiel, ce qui reste à faire, et le plan d'action priorisé pour rendre la plateforme performante.

GeoLight Service — SIG.tg SARL

Vitrine publique de la plateforme SIG.tg · sig-web-site (Django 5.2)

RÉFÉRENTIEL : CAHIER DES CHARGES SIG WEB SITE

PÉRIMÈTRE : CODE · SÉCURITÉ · RGPD · PERFORMANCE

16 SEPTEMBRE 2026

Table des matières

Synthèse exécutive	1
1. Contexte, objectifs et périmètre de l'audit	2
1.1 Contexte du projet	2
1.2 Objectifs de l'audit	2
1.3 Périmètre et documents de référence	2
1.4 Méthodologie et outils	3
2. Cartographie technique de l'existant	3
2.1 Stack technique observée contre stack prévue	3
2.2 Applications Django : prévues contre réelles	4
2.3 Qualité du dépôt Git et de la livraison	4
3. Conformité fonctionnelle au cahier des charges	5
3.1 Tableau d'avancement global	5
3.2 Écarts majeurs détaillés	6
3.3 Fonctionnalités techniques transverses	7
3.4 Livrables attendus	8
4. Audit de sécurité — OWASP Top 10 et RGPD	8
4.1 Grille OWASP Top 10	8
4.2 A05 — Durcissement de production (constat critique)	9
4.3 Conformité RGPD et protection des données	10
4.4 Vulnérabilités applicatives ciblées	11
4.5 Points forts de sécurité à préserver	11
5. Qualité du code et performance	12
5.1 Points forts à capitaliser	12
5.2 Dette technique identifiée	12
5.3 Performance et objectifs chiffrés	13
6. Plan d'action priorisé	13
6.1 Vue d'ensemble des constats	13
6.2 Actions critiques — à traiter avant toute mise en ligne	14
6.3 Actions majeures — nécessaires à l'exploitation conforme	14
6.4 Actions mineures — qualité et optimisation	15
6.5 Séquencement recommandé	16
7. Conclusion	17

Synthèse exécutive

Le présent rapport d'audit évalue la conformité du projet **SIG Web Site** (vitrine publique de la plateforme SIG.tg, développée par SIG.tg SARL / GeoLight Service) par rapport à son cahier des charges, et analyse l'état de sécurité et de qualité du code livré. L'audit a porté sur l'ensemble du dépôt « sig-web-site » : application Django 5.2 composée de sept applications internes, environ 2 200 lignes de tests automatisés, un historique Git de 205 commits, ainsi que sur la configuration de production et la base de données embarquée.

Le projet présente une base technique saine et des signaux de qualité réels : l'architecture Django est claire et modulaire, le référencement technique est soigné (django-meta, sitemaps, robots.txt, données structurées), l'internationalisation français/anglais est opérationnelle, et la suite de tests exécutée dans le cadre de l'audit affiche **221 tests réussis sur 221**. Ces acquis constituent un socle solide pour la suite.

En revanche, l'audit identifie **26 constats** dont 5 critiques qui empêchent une mise en production conforme au cahier des charges. Les plus graves concernent la conversion commerciale : le formulaire d'inscription n'envoie jamais les données au serveur (un script JavaScript affiche un écran de succès simulé, sans création de compte ni capture du lead), et la page Contact ne comporte aucun formulaire, alors que le cahier des charges exige un formulaire complet. S'ajoutent une configuration de production dépourvue de tous les réglages de sécurité (six alertes confirmées par la commande « `manage.py check --deploy` »), l'absence de bannière de consentement aux cookies et de Google Analytics, et un traitement d'historique de recherche non conforme au RGPD (adresses IP de visiteurs anonymes conservées sans base légale). Le taux de conformité fonctionnelle au cahier des charges est estimé à environ **63 %**.

221/221	6	5	≈ 63 %
tests automatisés réussis	alertes de sécurité (check --deploy)	écarts critiques au cahier des charges	conformité fonctionnelle estimée

Chiffres clés de l'audit — exécution des tests et vérifications du 16 septembre 2026

Le plan d'action proposé hiérarchise 26 corrections selon trois niveaux de gravité (5 critiques, 10 majeures, 11 mineures) et chiffre l'effort total entre **21 et 33 jours-homme**, soit trois à six semaines pour un développeur confirmé. Les cinq actions critiques (formulaires d'inscription et de contact, durcissement de la production, consentement cookies, conformité RGPD de la recherche) conditionnent la mise en ligne et doivent être traitées en priorité absolue.

1. Contexte, objectifs et périmètre de l'audit

1.1 Contexte du projet

SIG.tg SARL développe une plateforme cartographique en ligne bâtie sur des technologies open source (Django, PostgreSQL/PostGIS, OpenStreetMap), décomposée en deux entités : **SIG Platform**, l'espace métier sécurisé destiné aux utilisateurs authentifiés, et **SIG Web Site**, la vitrine publique chargée de présenter la plateforme, d'informer le public, de rassurer les organisations et de convertir les visiteurs en utilisateurs inscrits. Le cahier des charges audité (document de référence « CAHIER DES CHARGES – SIG WEB SITE », 7 pages) concerne exclusivement cette vitrine : onze pages principales, cinq applications Django prévues, un CMS via Django Admin, un déploiement sur VPS Contabo hébergé en Allemagne, pour un budget estimé de 12 800 € HT et un planning de neuf semaines.

À quelques semaines de la livraison prévisionnelle, l'équipe souhaite disposer d'un état des lieux objectif : ce qui est fait, ce qui est partiellement fait, ce qui reste à faire, les problèmes détectés et les correctifs à appliquer pour rendre la plateforme performante et sûre. Le présent rapport répond à cette demande en s'appuyant exclusivement sur des éléments vérifiables du dépôt livré.

1.2 Objectifs de l'audit

- Comparer le code livré au cahier des charges, page par page et fonctionnalité par fonctionnalité ;
- évaluer la sécurité applicative selon le référentiel OWASP Top 10 (2021) et la conformité RGPD ;
- apprécier la qualité du code, de la documentation et de la livraison (dépôt Git, dépendances, base de données) ;
- proposer un plan d'action priorisé par gravité, avec correctifs concrets et estimation d'effort.

1.3 Périmètre et documents de référence

Sont entrés dans le périmètre : le code source complet de « sig-web-site.zip » (hors environnement virtuel), le cahier des charges PDF, la base de données SQLite embarquée (db.sqlite3), le fichier d'environnement .env, la configuration Django (core/settings/base.py, dev.py, prod.py), les sept applications (pages, blog, contact, gallery, leads, search, newsletter), les gabarits HTML, les feuilles de style, les scripts JavaScript et l'historique Git (205 commits). Le code de la plateforme métier SIG Platform et les infrastructures d'hébergement réelles sont hors périmètre : l'audit ne peut pas attester de l'état effectif du VPS Contabo ni de la disponibilité du domaine sigtg.tech, ces points étant vérifiables uniquement lors d'une revue d'exploitation.

1.4 Méthodologie et outils

La méthode combine quatre techniques complémentaires. Premièrement, une revue manuelle approfondie du code (modèles, vues, formulaires, URLs, gabarits, réglages) croisée avec les exigences du cahier des charges. Deuxièmement, l'exécution de la suite de tests du projet dans un environnement Python 3.13 propre avec Django 5.2 et les dépendances du requirements.txt. Troisièmement, l'audit de configuration natif de Django (« python manage.py check --deploy » sous réglages de production) qui objectif les manques de sécurité. Quatrièmement, l'inspection directe de la base SQLite livrée (comptages par table, contrôle du hachage du compte administrateur). Chaque constat est rattaché à un fichier, priorisé par gravité (Critique / Majeur / Mineur) et, lorsqu'il est critique, accompagné d'un correctif proposé. Cette approche privilégie les preuves reproductibles plutôt que les impressions ; elle ne remplace toutefois pas un test d'intrusion complet, qui resterait recommandé avant la mise en production.

2. Cartographie technique de l'existant

2.1 Stack technique observée contre stack prévue

La stack réellement mise en œuvre est globalement conforme aux choix imposés par le cahier des charges, avec des écarts notables sur la base de données, l'envoi d'e-mails et l'absence de tout outillage de production. Le tableau suivant résume la comparaison.

Tableau 1 — Stack technique : prévisions du cahier des charges contre réalité du code

Composant	Prévu (CDC)	Observé dans le code	Verdict
Framework	Django 5.2+ (Python)	Django 5.2 confirmé (requirements.txt, venv 3.13.5)	OK
Base de données	PostgreSQL (SQLite toléré en dev)	SQLite par défaut partout ; aucune configuration PostgreSQL dans prod.py	Manquant
Front-end	HTML5, CSS3, JavaScript vanilla	Conforme — main.js (405 lignes) sans dépendance, design system CSS cohérent	OK
Icônes	Font Awesome 6+	Font Awesome 6.5.1 via CDN (sans attribut d'intégrité SRI)	Partiel
Cartographie	iframe OpenStreetMap (contact)	Carte Leaflet 1.9.4 + tuiles OSM — solution plus riche que prévu	OK
CMS	Django Admin	Conforme et étendu : sections PageSection administrables, context processors	OK
E-mails	SendGrid ou SMTP local	Backend « console » uniquement ; aucune intégration SendGrid/SMTP	Manquant
Hébergement	VPS Contabo, Nginx + Unicorn	Aucune trace dans le dépôt (ni Dockerfile, ni provisioning, ni CI/CD)	Manquant

Comparaison de la stack prévue et de la stack observée (source : requirements.txt, core/settings/, core/urls.py)

2.2 Applications Django : prévues contre réelles

Le cahier des charges prévoit cinq applications (pages, blog, legal, contact, newsletter). Le dépôt en contient sept, plus le projet « core » : les applications **gallery** (galerie de cartes publiques), **search** (recherche full-text) et **leads** (capture des demandes d'essai) sont des ajouts hors cahier des charges — des extensions pertinentes qu'il convient toutefois de faire valider par le donneur d'ordre comme évolutions contractuelles. À l'inverse, l'application **legal** prévue n'existe pas : les documents juridiques sont gérés dans « pages » (modèles ConditionOfUse, PrivacyAndSecurity, Rgpd, Cookies), ce qui est fonctionnellement équivalent mais doit être documenté. Cette réorganisation ne crée pas de problème technique, mais elle écarte le projet du découpage contractuel prévu.

Tableau 2 — Applications prévues et applications réelles

App. prévue (CDC)	Réalisation observée	Commentaire
pages	pages (12 modèles)	Étendue : héberge aussi les pages juridiques prévues dans l'app « legal »
blog	blog (Article, Category)	Conforme — SEO par article, archives, catégories
legal	absente (fusionnée dans pages)	Écart de structure, non de fonctionnalité — à documenter
contact	contact (ContactInfo)	Informations de contact uniquement — le formulaire exigé est absent
newsletter	newsletter (FormView + token de désinscription)	Conforme V1 — envoi d'e-mails et double opt-in restent à implémenter
(non prévu)	gallery — cartes publiques	Ajout hors CDC : à faire valider
(non prévu)	search — recherche full-text	Ajout hors CDC : à faire valider (point RGPD, cf. § 4.3)
(non prévu)	leads — demandes d'essai	Ajout hors CDC : logique serveur défaillante, cf. § 3.2

Correspondance entre le découpage applicatif prévu et le dépôt livré

2.3 Qualité du dépôt Git et de la livraison

L'historique Git (205 commits) témoigne d'un travail méthodique : branches « feature/ » et « fix/ » fusionnées dans « dev », messages de commit conventionnels (feat, fix, test, chore), résolution documentée d'un conflit de migrations (pages/0011) à la veille de la livraison. Le fichier .gitignore exclut correctement .env, db.sqlite3 et venv/ — ces fichiers ne sont donc pas versionnés. En revanche, l'archive zip livrée à l'auditeur contient tout de même le dossier venv (107 Mo, environnement Python incomplet — Django y est absent), la base de données de développement et le fichier .env avec une clé secrète de développement préfixée « django-insecure- ». Cette pratique expose des artefacts non contractuels et rend l'archive difficilement exploitable telle quelle ; le point est traité dans le plan d'action (m9). Par ailleurs, requirements.txt contient cinq doublons (asgiref, Django, django-enviro, pillow, sqlparse sont déclarés deux fois), ce qui révèle une construction incrémentale sans outil de verrouillage (aucun fichier de lock généré par pip-tools ou équivalent).

3. Conformité fonctionnelle au cahier des charges

3.1 Tableau d'avancement global

La figure 1 présente le taux de complétude estimé par module fonctionnel, en distinguant ce qui est terminé, ce qui est partiellement réalisé et ce qui manque. Le calcul pondère les exigences explicites du cahier des charges ; les ajouts hors périmètre (gallery, search) ne sont pas comptés dans le dénominateur. Le taux global de conformité ressort à environ 63 % : les pages de présentation sont quasiment achevées, tandis que les modules de conversion (contact, inscription, connexion, cookies/analytics) accusent les retards les plus importants — précisément ceux que le cahier des charges désigne comme objectifs commerciaux prioritaires.

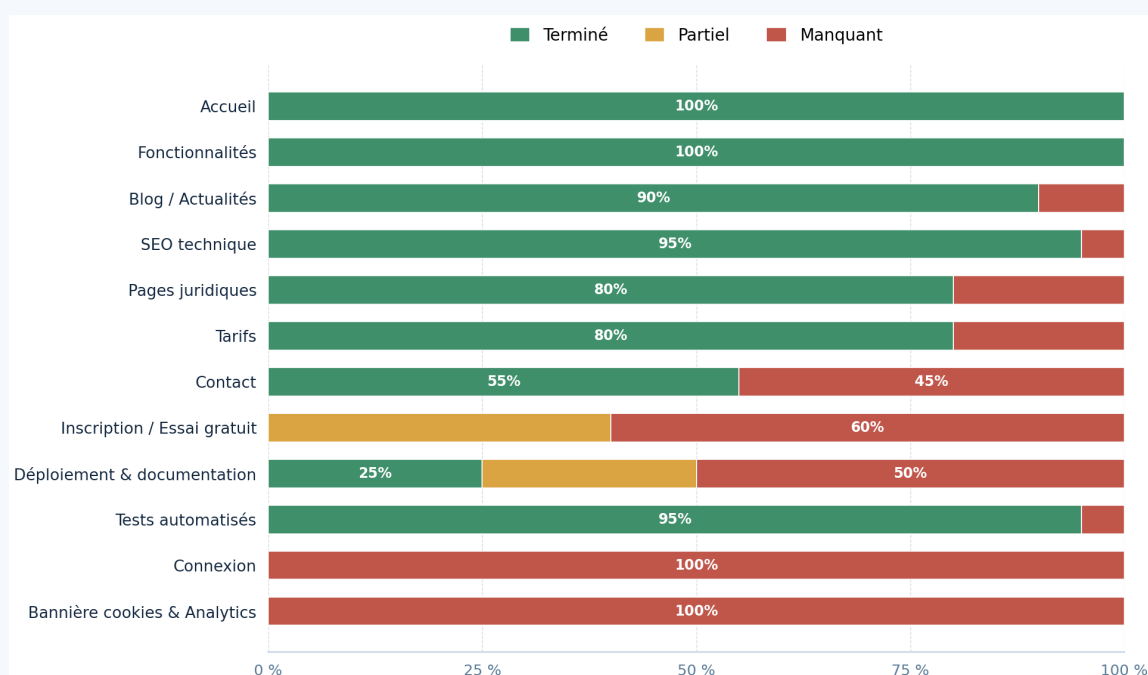


Figure 1 — Avancement par module fonctionnel (Terminé / Partiel / Manquant, estimation d'audit)

Tableau 3 — Les onze pages du cahier des charges : état observé

Page (CDC)	Statut	État observé dans le code	Écart
Accueil	Terminé	Hero éditable, 6 fonctionnalités, 4 chiffres clés, partenaires, étapes, témoignages, CTA — conforme	Aucun
Fonctionnalités	Terminé	5 modules détaillés + FAQ + illustrations éditables via l'admin	Aucun
Tarifs	Partiel	3 offres (Essentielle, Équipe, Institution) comparatives + FAQ ; le Pass Solidarité exigé est absent du gabarit et des fixtures	Majeur
Actualités	Terminé	Blog dynamique, catégories, archives, pagination (9), sidebar articles récents ; formulaire newsletter en sidebar absent (page dédiée à la place)	Mineur
Contact	Manquant	Coordonnées éditables, réseaux sociaux et carte Leaflet OSM conformes ; AUCUN formulaire de contact (exigence explicite : nom, prénom, organisation, e-mail, téléphone, type de structure, objet, message, case RGPD)	Critique
Mentions légales	Manquant	Vue LegalView et gabarit legal.html existent, mais l'URL est commentée dans pages/urls.py et aucun lien n'y pointe	Majeur
Politique de confidentialité	Terminé	privacy.html + modèle PrivacyAndSecurity + sections éditables	Aucun
CGU	Terminé	gtcu.html + modèle ConditionOfUse, contenu structuré éditable	Aucun
Politique des cookies	Partiel	cookies.html + modèle Cookies conformes ; mais aucune bannière de consentement réelle n'existe sur le site	Critique
Connexion	Manquant	Aucune page de connexion, aucun bouton « Se connecter » ; une URL de plateforme (PLATFORM_LOGIN_URL) est prévue mais jamais utilisée	Critique
Inscription / Essai gratuit	Manquant	Formulaire riche et soigné en apparence, mais sans envoi serveur (cf. § 3.2) ; ni mot de passe, ni création de compte	Critique

État des onze pages exigées — constats sourcés (gabarits, vues, fixtures, base livrée)

3.2 Écarts majeurs détaillés

3.2.1 — Le formulaire d'inscription n'envoie jamais les données au serveur

C'est le constat le plus grave de l'audit. La page « Essayer gratuitement » (leads/templates/leads/signup.html) présente un formulaire complet en trois sections (organisation, contact, besoins) avec acceptation des CGU. Mais le gabarit ne définit ni attribut « action » ni jeton CSRF, et son script JavaScript intercepte la soumission par « e.preventDefault() » puis affiche un écran de succès purement local : **aucune donnée n'est transmise, aucun compte n'est créé, aucun lead n'est enregistré**. L'écran de succès promet pourtant à l'utilisateur qu'un e-mail récapitulatif a été envoyé aux administrateurs — ce qui est factuellement faux et constitue un risque d'image sérieux. Côté serveur, la vue signup_view (leads/views.py) attend des champs « nom », « email » et « organisation » qui ne correspondent pas aux noms réels du gabarit (« fullname », « organization ») : même si le formulaire était soumis, la capture serait incomplète. Le cahier des charges demandait par ailleurs un champ mot de passe et une création de compte avec redirection vers SIG Platform ; la refonte actuelle (activation manuelle sous 24 h ouvrées) est un choix de produit défendable, mais elle n'est documentée nulle part et reste inopérante faute de soumission réelle. Toute l'acquisition de leads du site est donc inactive.

3.2.2 — La page Contact est dépourvue de formulaire

Le cahier des charges exige un formulaire de contact complet (nom, prénom, organisation, e-mail, téléphone, type de structure, objet, message, case à cocher RGPD) avec notification e-mail. Le gabarit `contact.html` n'affiche que les coordonnées administrables (modèle `ContactInfo`), un encart « demander une démo » renvoyant vers un simple lien `mailto`, les réseaux sociaux et la carte `OpenStreetMap`. Aucun modèle de message (`ContactRequest`), aucun formulaire, aucune vue de traitement POST n'existe dans l'application contact — les tests présents ne couvrent d'ailleurs que l'affichage des informations. L'exigence fonctionnelle la plus élémentaire d'une vitrine commerciale n'est donc pas implémentée.

3.2.3 — Connexion, bannière cookies et Google Analytics absents

Trois fonctionnalités techniques exigées sont totalement absentes. La page « Connexion » (e-mail, mot de passe, « se souvenir de moi », mot de passe oublié) n'a aucun équivalent : le site ne comporte aucun point d'entrée vers l'espace client, alors que le cahier des charges en fait un bouton du menu principal. La « bannière de consentement dès la première visite, avec paramétrage des préférences » n'existe ni dans `base.html` ni dans les dépendances (`django-cookie-consent` absent du `requirements.txt`) — un manquement RGPD autant que fonctionnel. Enfin, l'intégration Google Analytics « avec consentement » est absente, ce qui prive l'équipe des indicateurs de pilotage exigés (taux de conversion, rebond, visiteurs uniques).

3.2.4 — Mentions légales désactivées et Pass Solidarité absent

Deux écarts ponctuels mais explicites complètent le tableau. La route « `legal/` » est commentée dans `pages/urls.py` (ligne 10), rendant la page Mentions légales inaccessible alors que la vue et le gabarit existent : une décision probablement transitoire jamais révisée. Le Pass Solidarité destiné aux petites structures, exigé dans la page Tarifs, n'apparaît ni dans le gabarit `pricing.html` ni dans les fixtures `pricing_tiers.json` qui ne contiennent que les trois offres payantes.

3.3 Fonctionnalités techniques transverses

Tableau 4 — Fonctionnalités techniques exigées : état observé

Exigence (CDC § 3.2)	Statut	Détail
Menu de navigation responsive	Terminé	Burger mobile, overlay de recherche, sélecteur de langue FR/EN avec CSRF, CTA unique « s'abonner » (le bouton « Se connecter » exigé manque)
Sections modulaires via Django Admin	Terminé	PageSection administrable pour 10 pages + context processors header/footer — point fort du projet
Footer complet	Partiel	4 colonnes + liens juridiques + copyright éditable ; réseaux sociaux et lien Mentions légales absents
Gestion des cookies (bannière)	Manquant	Aucune bannière, aucune dépendance de consentement
SEO optimisé	Terminé	django-meta (Open Graph, Twitter, Schema.org), 3 sitemaps, robots.txt, canoniques, JSON-LD — niveau remarquable
Analytics (Google Analytics avec consentement)	Manquant	Aucun traceur, aucun mécanisme de consentement
Responsive design mobile-first	Terminé	Grilles fluides, media queries systématiques
Accessibilité WCAG 2.1 AA	Partiel	aria-labels, focus-visible et classes sr-only présents ; skip-link absent, emojis utilisés comme icônes, contrastes non audités

3.4 Livrables attendus

Tableau 5 — Les huit livrables du cahier des charges (§ 6)

Livable exigé	Statut	Constat
1. Code source sur dépôt Git	Terminé	Dépôt Git présent, 205 commits, .env et base exclus du versionnement
2. Documentation technique (README, installation, déploiement)	Partiel	README exhaustif mais avec artefacts (« Voici un README.md complet... » en tête de fichier), badges obsolètes (Django 4.2, Celery, PostGIS jamais utilisés) ; guide de déploiement Contabo non livré
3. Documentation administrateur	Manquant	Aucun guide d'administration du contenu
4. Maquettes UI/UX (Figma/XD)	Non vérifiable	Absentes du livrable — à fournir ou référencer
5. Base de données (schéma + fixtures)	Partiel	Migrations complètes, mais base livrée quasi vide (0 article, 0 tarif, 0 coordonnée) : les fixtures existantes ne sont pas chargées
6. Tests fonctionnels + rapport de recette	Partiel	221 tests unitaires/fonctionnels réussis (exécutés par l'audit) ; aucun rapport de recette, aucun test cross-browser documenté
7. Site en production (sigtg.tech)	Non vérifiable	Hors périmètre du dépôt — à attester lors d'une revue d'exploitation
8. Procédure de sauvegarde/restauration	Manquant	Aucune procédure documentée dans le dépôt

Suivi des livrables contractuels — état constaté au 16 septembre 2026

4. Audit de sécurité — OWASP Top 10 et RGPD

4.1 Grille OWASP Top 10

L'analyse a couvert les dix catégories du Top 10 OWASP (2021). Le bilan global est meilleur que ne le laisse craindre l'état de la configuration : les fondations Django sont bien exploitées (ORM systématique, CSRF actif, hachage robuste), mais la couche de durcissement de production est absente et deux traitements de données personnelles posent des problèmes de conformité RGPD.

Tableau 6 — Couverture OWASP Top 10 (2021)

Catégorie OWASP	Verdict	Constats principaux
A01 — Contrôle d'accès défaillant	Partiel	Admin Django exposé sans limitation de débit ni 2FA ; Rosetta (édition des traductions via le Web) monté en production ; cartes « archived » accessibles par URL directe (GalleryDetailView sans filtre)
A02 — Défaillances cryptographiques	Partiel	Hachage pbkdf2_sha256 (1 000 000 d'itérations) correct ; aucun secret en dur dans le code ; mais HSTS, redirection HTTPS et cookies sécurisés non configurés (cf. § 4.2)
A03 — Injection	Bon	ORM exclusivement (aucune requête brute) ; auto-échappement actif ; filtre linebreaks protège le contenu juridique ; résidus mineurs : jokers LIKE non neutralisés dans la recherche, code mort ContactInfo.get_rendered_content()
A04 — Conception non sécurisée	Partiel	Inscription simulée côté client (§ 3.2.1) ; aucun anti-spam (honeypot, captcha, rate limit) sur newsletter et recherche ; désinscription newsletter sans confirmation
A05 — Mauvaise configuration	Manquant	Six alertes « check --deploy » : HSTS, redirection SSL, cookies de session/CSRF, ALLOWED_HOSTS, clé secrète faible ; Rosetta exposé (cf. § 4.2)
A06 — Composants vulnérables	Partiel	Django 5.2 récent ; pas de fichier de verrouillage ni d'analyse SCA (pip-audit) intégrée ; dépendances non gelées intégralement
A07 — Défaillances d'authentification	Partiel	Validateurs de mots de passe actifs ; aucun verrouillage/limitation sur l'admin ; la fonctionnalité « mot de passe oublié » prévue au CDC est absente
A08 — Défaillances d'intégrité logicielle	Partiel	Ressources tierces (cdnjs, unpkg, Google Fonts) sans attribut d'intégrité SRI ; pas de chaîne CI/CD
A09 — Journalisation et surveillance	Manquant	Aucune configuration LOGGING ; aucun outil de suivi d'erreurs (Sentry) ; impossible d'investiguer un incident
A10 — SSRF	Bon	Aucune récupération d'URL côté serveur

Grille OWASP Top 10 appliquée au dépôt sig-web-site

4.2 A05 — Durcissement de production (constat critique)

Le fichier `core/settings/prod.py` se limite à trois lignes effectives : `import` du base, `DEBUG` à `False` et `ALLOWED_HOSTS` lu dans l'environnement (par défaut vide). Aucun réglage de sécurité n'y figure. Exécuté sur cette configuration, l'outil officiel « `manage.py check --deploy` » émet **six avertissements de sécurité** (`security.W004`, `W008`, `W009`, `W012`, `W016`, `W020`) : absence de HSTS, absence de redirection HTTPS, cookies de session et CSRF sans l'attribut `Secure`, `ALLOWED_HOSTS` vide et clé secrète faible (celle du `.env` livré). En l'état, la production serait vulnérable au dégradation de connexion (downgrade), à l'interception de session sur réseau non fiable et à l'empoisonnement du cache d'un hébergeur virtuel multi-domaines. Le correctif ci-dessous, à insérer dans `prod.py`, traite l'intégralité de ces alertes.

Correctif proposé — `core/settings/prod.py` (durcissement complet)

```

from .base import *

DEBUG = False
ALLOWED_HOSTS = env.list('DJANGO_ALLOWED_HOSTS') # obligatoire, ex. ['www.sigtg.tech']

# --- HTTPS / HSTS ---
SECURE_SSL_REDIRECT = True
SECURE_HSTS_SECONDS = 31536000 # 1 an (démarrer a 3600 en pilote)
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https') # derriere Nginx

# --- Cookies ---
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SESSION_COOKIE_AGE = 1209600 # 14 j, ajustable via 'Se souvenir de moi'

# --- En-tetes de reponse ---
SECURE_CONTENT_TYPE_NOSNIFF = True
SECURE_REFERRER_POLICY = 'strict-origin-when-cross-origin'
X_FRAME_OPTIONS = 'DENY'

# --- Outils de developpement interdits en production ---
if not DEBUG:
    INSTALLED_APPS = [a for a in INSTALLED_APPS if a != 'rosetta']

```

Code 1 — Réglages de production à appliquer ; ils font disparaître les six alertes W004/W008/W009/W012/W016/W020

Deux points complémentaires : la clé secrète doit être régénérée (50 caractères aléatoires minimum) et stockée uniquement dans l'environnement du serveur — celle du .env livré, préfixée « django-insecure- », doit être considérée comme compromise ; et l'application rosetta, qui permet d'éditer les fichiers de traduction depuis le Web, doit être retirée des installations de production (elle est actuellement montée sans condition dans core/urls.py).

4.3 Conformité RGPD et protection des données

Deux traitements sont directement problématiques. Premièrement, la vue de recherche (search/views.py) enregistre systématiquement dans la table search_searchhistory la requête et l'adresse IP de tout visiteur, y compris anonyme : l'IP est une donnée personnelle au sens du RGPD (CJUE, arrêt Breyer), et ce stockage sans information préalable, sans base légale documentée ni durée de conservation expose l'entreprise. Deuxièmement, l'absence de bannière de consentement (§ 3.2.3) signifie qu'aucune base légale valide ne pourrait encadrer un jour l'activation d'outils de mesure d'audience — le couple « politique cookies sans mécanisme de consentement » n'est pas défendable en cas de contrôle. La redirection d'inscription prévue côté serveur transmettait par ailleurs nom, e-mail et organisation en paramètres d'URL (visibles dans les journaux et l'historique du navigateur) : le correctif de la vue d'inscription doit remplacer ce mécanisme par un envoi en POST ou par une transmission côté serveur (jeton signé). Enfin, il est recommandé de documenter la rétention appliquée à l'historique de recherche dans la politique de confidentialité et de purger le contenu déjà accumulé en base de développement.

Correctif proposé — search/views.py (anonymisation et rétention)

```

import hashlib
from django.conf import settings
from django.utils import timezone

def _pseudo_ip(ip: str) -> str:
    """Pseudo-anonymise l'IP (RGPD art. 32), cle tournante mensuelle."""
    salt = timezone.now().strftime('%Y-%m')
    return hashlib.sha256(f"{settings.SECRET_KEY}:{salt}:{ip}".encode()).hexdigest()[:16]

# Dans SearchView.get() :
if len(query) <= 120: # borne la longueur stockee
    SearchHistory.objects.create(
        query=query[:120],
        ip_address=None, # ne plus stocker l'IP en clair
        ip_hash=_pseudo_ip(self.get_client_ip(request)),
    )

# Purge periodique (commande de gestion + cron) :
# SearchHistory.objects.filter(created_at__lt=timezone.now()-timezone.timedelta(days=30)).delete()

```

Code 2 — Pseudo-anonymisation immédiate et purge à 30 jours ; alternative recommandée : ne journaliser qu'un compteur agrégé de recherches

4.4 Vulnérabilités applicatives ciblées

- **ContactInfo.get_rendered_content()** (`contact/models.py`) : transmet le contenu administrable comme chaîne de format à `format_html()` — une accolade « { } » dans le contenu provoque une erreur 500, et le contenu n'est pas échappé quand il est passé en premier argument. Le code est actuellement mort (le gabarit utilise `info.content|linebreaks`) mais doit être corrigé ou supprimé ;
- **SearchAutocompleteView** (`search/views.py`) : `int(request.GET.get('limit', 10))` sans garde — une valeur non numérique déclenche une erreur 500 exploitable ; la recherche `icontains` laisse passer les jokers `%` et `_` (dénier de service léger sur les requêtes) ;
- **get_client_ip()** : fait aveuglément confiance à l'en-tête `X-Forwarded-For` (usurpable) — acceptable uniquement derrière un reverse proxy qui l'écrase ;
- **GalleryDetailView** : ne filtre pas `archived=False` — les cartes retirées restent accessibles par URL directe (fuite d'information mineure) ;
- **Ressources tierces sans SRI** : Font Awesome (cdnjs), Leaflet (unpkg) et les polices Google sont chargées sans attribut d'intégrité ; l'auto-hébergement de ces actifs (action m1) règle également la conformité RGPD des polices Google.

4.5 Points forts de sécurité à préserver

L'audit tient à souligner les acquis : usage exclusif de l'ORM (aucune requête SQL brute dans tout le dépôt), middleware CSRF actif avec jetons présents dans les formulaires qui en ont besoin (changement de langue, newsletter), hachage des mots de passe conforme aux défauts robustes de Django 5.2, secrets gérés par variables d'environnement via `django-environ` et exclus du versionnement, génération du jeton de désinscription newsletter par `secrets.token_urlsafe(32)`, et filtres de requêtage systématiques (contenus non publiés et archivés jamais exposés sur le blog). Ces bonnes pratiques, appliquées de manière homogène par l'équipe, réduisent sensiblement la surface de risque et doivent être conservées telles quelles.

5. Qualité du code et performance

5.1 Points forts à capitaliser

La qualité générale du code est au-dessus de la moyenne des projets de ce gabarit. La suite de tests, exécutée dans un environnement propre par l'audit, compte **221 tests tous réussis en 1,3 seconde** (environ 2 200 lignes réparties en `test_models.py` et `test_views.py` par application), et la vérification système de Django ne remonte aucune anomalie. L'architecture est homogène : vues génériques Django, mixins SEO systématiques, modèles commentés avec méthodes métier, context processors propres pour le header et le footer éditables. L'internationalisation français/anglais est réellement en place (django-modeltranslation sur les contenus, fichiers `.po/.mo` compilés pour les deux langues, sélecteur de langue fonctionnel). Le JavaScript vanilla (405 lignes) est structuré par attributs `data-*`, sans dépendance externe, avec des comportements documentés en tête de fichier. Enfin, la charte technique du cahier des charges (vert #1a7f5c, bleu marine #1e3b5c, orange #f39c12, mobile-first) est respectée par les feuilles de style.

5.2 Dette technique identifiée

- **requirements.txt** : cinq dépendances déclarées deux fois, aucun fichier de verrouillage (pip-tools ou équivalent) ni d'analyse de vulnérabilités des composants ;
- **README.md** : commence par un artefact de génération (« Voici un README.md complet et professionnel... »), affiche un badge Django 4.2 alors que le projet tourne en 5.2, et documente Celery et PostGIS qui ne sont utilisés nulle part ;
- **Code mort** : `LegalView` orpheline (URL commentée), `ContactInfo.get_rendered_content()` défaillant (§ 4.4), gabarit `gallery/list.html` redondant avec `gallery_list.html` ;
- **Migrations** : onze migrations pages dont une fusion de conflit datant de la veille de la livraison — l'historique mériterait un compactage (squash) avant la mise en production ;
- **Base livrée vide** : aucune fixture chargée (0 article, 0 tarif, 0 coordonnée de contact, 0 carte) alors que les fichiers existent dans `pages/fixtures/` et `blog/fixtures/` ; le site livré s'affiche donc quasi vide et ne permet pas une recette client immédiate ;
- **Livrable zip pollué** : `venv` incomplet de 107 Mo, `db.sqlite3` de développement et `.env` inclus dans l'archive ; le `.git` est également embarqué.

5.3 Performance et objectifs chiffrés

Le cahier des charges fixe des objectifs mesurables : chargement inférieur à 2 secondes, score Lighthouse supérieur à 90, disponibilité de 99,9 %. Le code actuel n'instaure aucun des mécanismes habituels pour les atteindre en production : pas de middleware de compression (GZip), pas de stockage manifeste pour l'empreinte des fichiers statiques (ManifestStaticFilesStorage), pas de configuration de cache (CACHES absent — les requêtes de context processors s'exécutent à chaque affichage), pas d'attribut de chargement différé sur les images, et aucun outil d'optimisation d'images (django-imagekit prévu à l'issue ISS-025 du plan projet n'a jamais été intégré). Les ressources tierces (Google Fonts, Font Awesome, Leaflet) proviennent de trois CDN distincts, ce qui ajoute des allers-retours DNS/TLS et des dépendances de disponibilité externes. Le point positif : les requêtes principales utilisent `select_related` et des index sont posés sur les champs de filtrage ; le détail d'article charge toutefois l'intégralité de la catégorie en mémoire pour calculer les liens précédent/suivant, un point à optimiser dès que le blog grossira. Aucun score Lighthouse ne peut être certifié par cet audit documentaire : une mesure sur l'environnement de production reste à programmer (action m11 du plan).

Tableau 7 — Indicateurs cibles du cahier des charges (§ 9) et état actuel

Indicateur (CDC)	Cible	État constaté
Temps de chargement des pages	< 2 s	Non mesuré — aucun outillage de performance en place
Score Lighthouse (performance)	> 90/100	Non mesuré — audit documentaire, mesure à programmer
Taux de rebond	< 50 %	Non mesurable — Google Analytics absent
Visiteurs uniques mensuels	5 000+	Non mesurable — Google Analytics absent
Taux de conversion vers l'inscription	> 5 %	Non mesurable — et l'inscription ne capture aucun lead (§ 3.2.1)
Accessibilité	WCAG 2.1 AA	Partiellement préparée (ARIA, focus-visible) ; audit WAVE/axe à réaliser
Disponibilité (uptime)	99,9 %	Non garantissable — aucun monitoring, aucune alerte dans le dépôt

Indicateurs de performance exigés et capacité actuelle à les mesurer

6. Plan d'action priorisé

6.1 Vue d'ensemble des constats

Vingt-six constats ont été consignés, répartis en 5 critiques (mettant en échec des exigences explicites du cahier des charges ou la sécurité de production), 10 majeurs (nécessaires à une exploitation conforme et sereine) et 11 mineurs (qualité, hygiène et optimisation). La figure 2 illustre cette répartition. Chaque action est estimée en taille d'effort : S (demi-journée au plus), M (un à deux jours), L (trois jours et plus), en développeur confirmé Django.

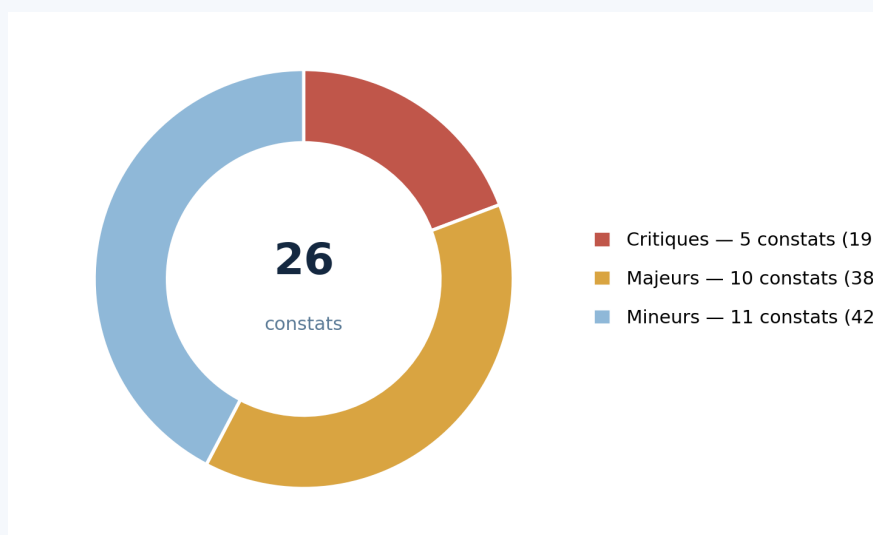


Figure 2 — Répartition des 26 constats par gravité

6.2 Actions critiques — à traiter avant toute mise en ligne

Tableau 8 — Actions critiques (C1 à C5)

N°	Action	Module	Effort
C1	Brancher le formulaire d'inscription : Django Form + jeton CSRF, validation serveur, enregistrement du lead (aligner les noms de champs), écran de succès servi par le serveur, transmission POST vers SIG Platform (supprimer les données personnelles de la chaîne de requête)	leads	L
C2	Créer le formulaire de contact : modèle ContactRequest (nom, prénom, organisation, e-mail, téléphone, type, objet, message, consentement RGPD), vue POST, notification e-mail aux administrateurs, accusé de réception	contact	M
C3	Durcissement de production : appliquer le correctif prod.py (Code 1), régénérer la clé secrète, définir DJANGO_ALLOWED_HOSTS, retirer rosetta, configurer LOGGING + Sentry	core/settings	S
C4	Bannière de consentement cookies (django-cookie-consent ou équivalent) avec trois choix (accepter, paramétrer, refuser), blocage de tout script tiers avant consentement, journalisation du choix	RGPD / front	M
C5	Conformité RGPD de la recherche : pseudo-anonymisation des IP (Code 2), purge à 30 jours, information dans la politique de confidentialité	search	S

Actions critiques — effort cumulé estimé : 7 à 11 jours-homme

6.3 Actions majeures — nécessaires à l'exploitation conforme

Tableau 9 — Actions majeures (M1 à M10)

N°	Action	Module	Effort
M1	Page Connexion : vue de redirection vers PLATFORM_LOGIN_URL (bouton « Se connecter » dans le header) ; si l'espace client n'existe pas encore, page d'information dédiée	pages	S
M2	Réactiver la page Mentions légales : décommenter la route legal/, ajouter le lien dans le footer	pages	S
M3	E-mails transactionnels réels : SendGrid ou SMTP, réglages par environnement, notification contact, confirmation newsletter, contenu du gabarit d'accueil des leads	core / e-mail	M
M4	PostgreSQL en production : moteur psycopg, DATABASE_URL, revérifier la recherche plein texte (les fonctions trigram exigent PostgreSQL) et les migrations	BDD	M
M5	Charger les fixtures de démonstration et saisir le contenu éditorial du CDC (5 articles de lancement, tarifs, chiffres clés, coordonnées)	contenu	S
M6	Ajouter le Pass Solidarité (quatrième PricingTier + encart page Tarifs)	tarifs	S
M7	Anti-spam global : honeypot + django-ratelimit sur newsletter, contact et recherche	sécurité	S
M8	Chaîne de déploiement : provisioning VPS Contabo (Nginx, Unicorn systemd, certbot), Dockerfile optionnel, CI/CD GitHub Actions (tests, migrations, collectstatic, reload)	devops	L
M9	Documentation : guide administrateur (Django Admin), guide de déploiement, procédure de sauvegarde/restauration (pg_dump + test de restauration)	docs	M
M10	Verrouiller les dépendances : dédoublonner requirements.txt, générer un fichier de lock (pip-tools) et intégrer pip-audit dans la CI	qualité	S

Actions majeures — effort cumulé estimé : 9 à 14 jours-homme

6.4 Actions mineures — qualité et optimisation

Tableau 10 — Actions mineures (m1 à m11)

N°	Action	Module	Effort
m1	Auto-héberger polices, Font Awesome et Leaflet (vendor static) + attribut d'intégrité sur tout CDN restant	front	M
m2	Performance : Whitenoise + ManifestStaticFilesStorage, GZipMiddleware, CACHES (Redis), chargement différé des images, miniatures blog	perf	S
m3	Accessibilité : lien d'évitement (skip-link), remplacer les emojis du header/carte par des icônes, audit de contraste WAVE/axe + correction	front	M
m4	Nettoyer le README (retirer l'artefact d'en-tête, corriger badges et stack documentée), documenter le choix « activation manuelle » de l'inscription	docs	S
m5	Purger le code mort : LegalView (après M2, supprimer si inutile), get_rendered_content (correction format_html ou suppression), gabarit gallery/list.html	qualité	S
m6	GalleryDetailView : filtrer archived=False	gallery	S
m7	Robustesse recherche : garde try/except sur limit, neutralisation des jokers LIKE, X-Forwarded-For accepté uniquement derrière proxy configuré	search	S
m8	Blog précédent/suivant : requêtes limitrophes plutôt que chargement complet de la catégorie en mémoire	blog	S
m9	Hygiène des livrables : exclure venv/, db.sqlite3, .env et .git des archives de recette (script de packaging)	qualité	S
m10	Compactage des migrations pages (squash) avant mise en production	BDD	S
m11	Mesure Lighthouse + accessibilité sur l'environnement de recette, et suivi des KPI du CDC une fois l'analytics actif	qualité	S

Actions mineures — effort cumulé estimé : 5 à 8 jours-homme

6.5 Séquencement recommandé

Séquence 1 (semaine 1) : C3 d'abord — une demi-journée de configuration réduit immédiatement le risque maximal ; puis C1 et C2, qui mobilisent l'essentiel de la semaine. Séquence 2 (semaine 2) : C4, C5, puis M2, M1, M5 et M6, qui rendent le site conforme sur le plan juridique et complet côté contenu ; M3 s'y greffe naturellement dès que le formulaire de contact existe. Séquence 3 (semaines 3 et 4) : M4, M8, M9, M10 et les actions mineures, avant une recette finale complète (tests cross-browser, Lighthouse, accessibilité) sur l'environnement de production. L'effort total estimé — 21 à 33 jours-homme — tient dans le budget de trois à six semaines à un développeur, ou deux semaines à deux développeurs en parallèle si C1/C2 et C4/C5 sont menés simultanément. Ce chiffrage n'inclut pas la partie plateforme métier (SIG Platform) ni les maquettes manquantes, qui doivent être clarifiées avec le donneur d'ordre.

7. Conclusion

Le SIG Web Site n'est ni un projet à recommencer ni un produit prêt à livrer : c'est un projet à **63 % conforme**, dont la partie vitrine et référencement — souvent la plus coûteuse — est largement réussie, tandis que la couche de conversion et la couche d'exploitation sécurisée restent à construire. Les fondations sont bonnes : architecture Django claire, tests systématiques, CMS modulaire véritablement utilisé, internationalisation fonctionnelle, et des pratiques de développement (Git, conventions, revue par branches) qui inspirent confiance pour la suite.

Le verdict de l'audit est donc un « **sous condition** » : la mise en production ne devrait être prononcée qu'après l'achèvement des cinq actions critiques (C1 à C5) — à savoir le branchement réel du formulaire d'inscription, la création du formulaire de contact, le durcissement complet de la configuration de production, l'installation du mécanisme de consentement aux cookies et la mise en conformité RGPD de l'historique de recherche. Les dix actions majeures complètent ensuite la conformité contractuelle (connexion, mentions légales, e-mails, PostgreSQL, Pass Solidarité, déploiement documenté). Avec le plan d'action fourni — correctifs de code à l'appui, l'équipe dispose de tout le nécessaire pour transformer en quelques semaines ce socle prometteur en une vitrine à la hauteur des objectifs commerciaux fixés par le cahier des charges : séduire, informer, rassurer et convertir.